DBMS

Assignment - 3

Date

Name - Deepak
Roll No - 2401010085
B.Tech CSE Core

Section - A

Q.1 :- What is SQL ? Why is it important in DBMS ?
=> Structured query language is a stardard language used to interact with databases.

~~Role of~~

Importance in DBMS :-
- It is the core language for managing data.
- Helps in efficient data retrival.
- Supports data integrity & Security.

Q.2 :- Difference b/w DDL & DML ?
=>

Feature	DDL	DML
Purpose	Defines Structure	Manipulates data.
Commands	CREATE, ALTER, DROP	INSET, Update, Delete.
Auto Commit	Yes	No

Q.3 :- What is JOIN ? Explain its types
→ Join is used to combine data from multiple tables based on a related column.

Types of JOIN :-
i) Inner Join :- returns matching records from both tables.
ii) Left Join :- returns all records from left table + matched from right.
iii) Right Join :- returns all records from right table + matched from left.
iv) Full Join :- returns all records from both tables

Spiral

Date.....

Q.4 :- Group By and Having :-

=> Group By : Grouped rows with same values (used with count, sum, avg)

Having : Filtered grouped data (used after Group By)

example ;

```
SELECT department, COUNT(*)
FROM employees
GROUP BY department
HAVING COUNT(*) > 5;
```

Q.5 :- Subquery :-

=> A query inside another query.

example ;

```
SELECT name
FROM employees
WHERE salary > (SELECT AVG(salary) FROM employees);
```

Types :-

Single - row.

Multi - row.

Correlated.

Q.6 :- Relational Algebra :-

=> A procedural query language from DBMS.

Works on tables (relations).

Operations :

σ (Selection), π (Projection), ∪ (Union), ∩ (Intersection), - (Difference), Join.

Traditional Theory Date.....

Q.7 :- where vs Having :-

2)

WHERE : Filters rows (before GROUP BY), no aggregate functions

HAVING: Filters groups (after GROUP BY, allows aggregate functions.

Section- B

1. Students with marks > 75.

```
SELECT *
FROM students
WHERE marks > 75;
```

2. JOIN Student & Course tables.

```
SELECT students.name, course.course_name
FROM students
INNER JOIN course
ON students.course_id = course.course_id;
```

3. Count students per department

```
SELECT department, COUNT(*) AS total_students
FROM students
GROUP BY departments;
```

Section- C

1. Employees in a specific department.

```
SELECT e.name
FROM employees e
```

Spiral

Date.....

```
Join departments d
ON e.dept_id = d.dept_id
WHERE d.dept_name = 'HR';
```

2. Total sales per customer :-

```
SELECT customer_id, SUM(amount) AS total_sales
FROM orders
GROUP BY customer_id;
```

3. Top 3 students :-

```
SELECT name, marks
FROM students
ORDER BY marks DESC
LIMIT 3;
```

Section - 2

1. Real-life SQL query + Logic :-

```
SELECT name, salary
FROM employees
WHERE salary > 5000;
```

2. JOIN query + use case :-

```
SELECT orders.order_id, customers.name
FROM orders
JOIN customers
ON orders.customer_id = customers.customer_id;
```

Spiral

Date.....

3. GROUP BY query + explanation:-

```
SELECT department, AVG(salary) AS avg_salary
FROM employees
GROUP BY department;
```